

Practice Exercise 1- Create and Manage a Database Custom Resource Definition (CRD) in Amazon EKS

Objective

The objective of this assignment is to extend the Kubernetes API by creating a custom resource type for managing databases. Students will design, deploy, validate, and manage database resources using Custom Resource Definitions (CRDs) and Custom Resources (CRs) in an Amazon EKS cluster.

Scenario

Your organization is building a cloud platform where application teams can request databases using Kubernetes-native APIs. Instead of manually documenting databases, the platform team wants every database to be represented as a Kubernetes Custom Resource. You have been asked to design and implement a new Kubernetes resource called **Database** that will store information such as:

- Database engine
 - Database version
 - Storage allocation
 - Number of replicas
 - Administrator username
 - Environment type
-

Assignment Tasks

Task 1: Explore Existing Kubernetes Extensions

1. Connect to the EKS cluster.
2. Verify cluster connectivity.
3. List all existing Custom Resource Definitions (CRDs).
4. Identify at least three CRDs already installed in the cluster.
5. Document the purpose of each CRD.

Deliverables

- Screenshot of cluster connectivity.
 - Screenshot of installed CRDs.
 - Brief explanation of three discovered CRDs.
-

Task 2: Design a Database Custom Resource Definition

Design a new CRD named **Database**.

Requirements:

1. Create a new API Group.
2. Define a Kind named **Database**.
3. Define singular and plural names.
4. Add a short name.
5. Decide whether the resource should be Namespaced or Cluster-scoped.
6. Create validation rules for mandatory fields.

Database Specification Must Include

- Database Engine
- Database Version

- Storage Size
- Replica Count
- Administrator Username
- Environment (Dev/Test/Prod)

Deliverables

- Architecture diagram of the CRD.
 - List of fields and their data types.
 - Justification for selected scope.
-

Task 3: Deploy and Verify the CRD

1. Deploy the Database CRD into the cluster.
2. Verify successful installation.
3. Inspect the CRD details.
4. Verify that Kubernetes recognizes the new API resource.

Deliverables

- Commands executed.
 - Screenshots showing successful CRD creation.
 - Evidence that the new API resource is available.
-

Task 4: Create Database Custom Resources

Create the following Database resources:

Database 1

Customer Management Database

Database 2

Inventory Management Database

Database 3

Payment Processing Database

Each resource must contain different values for:

- Database Engine
- Version
- Storage
- Replicas

Deliverables

- Resource definitions.
 - Screenshot showing successful creation.
-

Task 5: Query and Inspect Custom Resources

Perform the following operations:

1. List all Database resources.
2. Retrieve a specific Database resource.
3. Display resource details.

Deliverables

- Screenshots of outputs.
 - Brief explanation of observed metadata and specifications.
-

Task 6: Test Schema Validation

Create an invalid Database resource that violates the schema requirements.

Examples:

- Missing required fields
- Invalid data types
- Empty mandatory values

Observe the validation behavior.